

Objetivo del refactor

- Reducir el acoplamiento al SDK externo.
- Eliminar lógica distribuida en múltiples capas.
- Separar responsabilidades (Application vs Infrastructure).
- Mejorar la mantenibilidad y claridad estructural.
- Aumentar la testeabilidad.
- Reducir deuda técnica futura.
- Preparar el sistema para escalabilidad o eventual cambio de proveedor.

Estado actual VS Estado propuesto

Estado Actual

El código relacionado con Stream se encuentra distribuido en:

Core

- app/Providers/StreamServiceProvider.php
- app/Utilities/StreamGroupSyncHelper.php
- app/Utilities/TestStreamResponse.php

HTTP

- app/Http/Controllers/StreamController.php
- app/Http/Middleware/StreamEnabledForClient.php
- app/Http/Middleware/StreamWebhookGuardian.php

Jobs

- app/Jobs/Stream/*
- UpdateUserStreamGroups.php

Modelos

- StreamAutoMember
- StreamManualMember
- StreamLog

DTO

- app/DTO/Stream/*

Problemas detectados

Se tiene logica de negocio mezclada con integracion tecnica, hay usos de helpers estaticos (complejiza el testeo), hay dependencia directa del SDK de stream en multiples capas.

No existe una interfaz (contract) que abstraiga la mensajeria, hay alto acoplamiento a Laravel y no esta encapsulado como un módulo aislado.

Estado Propuesto

Se modularizará el sistema bajo un nuevo namespace: **app/Messaging**

Se propone esta nueva estructura (puede variar segun la arquitectura aprobada):

```
app/  
└─ Messaging/  
    └─ Domain/  
        └─ Contracts/  
            └─ MessagingGateway.php  
        └─ Application/  
            └─ UseCases/  
                └─ SyncUserGroups.php  
                └─ CreateChannel.php  
                └─ AddMembers.php  
                └─ HandleWebhook.php  
        └─ Infrastructure/  
            └─ Stream/  
                └─ StreamMessagingGateway.php  
                └─ StreamClientFactory.php  
    └─ Presentation/  
        └─ StreamController.php
```

VIRGILIO PADILLA - DEVELOPER

Que se refactorizará:

MessagingGateway (Interface)

- Abstracción completa del proveedor.
- El dominio no depende del SDK.

Encapsulamiento del SDK en StreamMessagingGateway

- Toda interacción con Stream se centraliza.
- Eliminación de uso directo del SDK en otras capas.

Eliminación de StreamGroupSyncHelper

- Conversión a SyncUserGroupsUseCase
- Eliminación de métodos estáticos.

Refactor de Controllers

- Remover lógica.
- Delegar a Application layer.

Ajuste del ServiceProvider

- Bind de interface a implementación concreta.
- Centralización de configuración.

Archivos impactados

Archivos Modificados:

- app/Providers/StreamServiceProvider.php
- app/Utilities/StreamGroupSyncHelper.php (eliminado)
- app/Http/Controllers/StreamController.php
- app/Jobs/Stream/*
- updateUserStreamGroups.php

Nuevos Archivos

- app/Messaging/Domain/Contracts/MessagingGateway.php
- app/Messaging/Application/UseCases/*
- app/Messaging/Infrastructure/Stream/StreamMessagingGateway.php
- app/Messaging/Infrastructure/Stream/StreamClientFactory.php

Riesgos Identificados

Romper sincronización de grupos

Mitigación:

- Mantener lógica intacta.
- Validar contra tests existentes.
- Testing manual en entorno staging.

Fallos en Webhooks

Mitigación:

- No modificar contratos externos.
- Tests específicos de webhook.
- Logging temporal ampliado durante deploy.

Dependencias ocultas no detectadas

Mitigación:

- Análisis previo exhaustivo.
- Búsqueda global de referencias al SDK.
- Refactor por etapas.

Impacto en Jobs en producción

Mitigación:

- Mantener firmas públicas.
- Deploy controlado
- Monitoreo post-release.

Resultado Esperado

- Módulo encapsulado y coherente.
- Código desacoplado del proveedor externo.
- Mayor claridad estructural.
- Tests más robustos.
- Reducción significativa de deuda técnica.
- Base preparada para escalabilidad futura.